

Concorrenza in Rust per l'esame

Schema concettuale, pattern pronti all'uso e catalogo di errori tipici per riconoscere e risolvere gli esercizi con thread, stato condiviso e strutture "thread-safe" — costruito a partire dall'esercizio Aggregator.

1. Come leggere il testo dell'esame
 2. Il framework mentale in 5 domande
 3. Le primitive fondamentali
 4. Sei pattern pronti all'uso
 5. Catalogo degli errori classici
 6. Checklist operativa passo-passo
 7. Esempio completo commentato — Aggregator
 8. Scheda di riferimento rapido
-

1. Come leggere il testo dell'esame

Questi esami seguono quasi sempre lo stesso schema: una struct con dei metodi che devono essere **thread-safe**, uno stato che viene aggiornato da più chiamanti, e spesso un thread interno che lavora in background. Il testo contiene sempre delle **parole-spia** che ti dicono, riga per riga, quale meccanismo Rust userai. Allenati a sottolinearle appena leggi il testo, prima ancora di guardare lo stub di codice.

FRASE NEL TESTO	COSA SIGNIFICA IN RUST
"più thread", "in modo asincrono", "concorrentemente"	Stato condiviso → <code>Arc<Mutex<T>></code> (o <code>RwLock</code>)
"offre metodi thread-safe"	I metodi prendono <code>&self</code> , non <code>&mut self</code> : la mutabilità passa dal <code>Mutex</code> / <code>Atomic</code> interno (interior mutability)
"un thread... si occupa di", "in background", "periodicamente", "ad intervalli regolari"	Thread interno creato in <code>new()</code> con <code>thread::spawn</code> , loop + <code>sleep</code> o timer
"alla distruzione della struttura... terminato in modo sicuro"	<code>impl Drop</code> : segnala lo stop e fai <code>join()</code> sul thread
"restituisce ... calcolato durante l'ultimo periodo"	Serve uno "snapshot" salvato altrove, non un ricalcolo al volo: due stati distinti (accumulo corrente / ultima fotografia)
"più thread possono inviare dati contemporaneamente"	Il test userà <code>thread::scope</code> con più <code>s.spawn</code> : il tuo <code>Mutex</code> deve reggere scritture concorrenti
"in coda", "produttore/consumatore", "notifica quando arriva"	<code>mpsc::channel</code> (invece di, o oltre a, <code>Mutex</code>) — vedi Pattern E
"aspetta finché non è pronto/disponibile"	<code>Condvar</code> — vedi Pattern F

CONSIGLIO PRATICO

Leggi i test **prima** di scrivere una riga di implementazione. In questi esami i test dello stub non sono un extra: sono la specifica precisa (con tanto di millisecondi di `sleep`) di quando un dato deve apparire, sparire, o restare invariato. Ogni ambiguità del testo si risolve leggendo cosa il test si aspetta esattamente.

2. Il framework mentale in 5 domande

Prima di scrivere codice, rispondi in ordine a queste domande. Ogni risposta ti porta dritto a una primitiva Rust precisa — è il "decision tree" da percorrere sempre allo stesso modo.

D1 C'è un dato che viene letto e scritto da più thread?

Sì, letture e scritture miste, frequenti → `Arc<Mutex<T>>`

Sì, ma è quasi sempre letto e raramente scritto → `Arc<RwLock<T>>` (più letture concorrenti ammesse)

Sì, ma è solo un contatore/flag semplice (bool, un numero) → `Arc<AtomicBool>` / `AtomicUsize` / `AtomicU64`, niente `Mutex`

NO, i thread non condividono dati ma si scambiano messaggi → `mpsc::channel`

D2 Serve un'attività periodica o "in background"?

Sì → un thread creato in `new()` con un loop; per lo stop scegli fra Pattern B1 (`Atomic`+`sleep`, semplice) e Pattern B2 (`channel`+`recv_timeout`, stop istantaneo)

NO → niente thread interno: i metodi pubblici bastano da soli

D3 La struct ha un `Drop` da scrivere?

Sì, appena c'è un thread interno → salva sempre l'handle come `Option<JoinHandle<()>>`; in `drop()` prima segnali lo stop, poi `.take().join()`

D4 I chiamanti devono aspettare un evento preciso (non solo "un po' di tempo")?

Sì → `Condvar` abbinata al `Mutex` dello stato (Pattern F)

NO, il testo/i test usano solo `sleep` → non serve `Condvar`, un semplice `Mutex` + thread periodico basta

D5 Devo lanciare più thread e raccogliere i loro risultati nello stesso scope, senza farli sopravvivere alla funzione?

Sì → `thread::scope` (Pattern D): eviti `Arc`, puoi prendere in prestito dati locali

NO, i thread devono vivere più a lungo della funzione che li crea → `thread::spawn` "puro" con `Arc` + `move` (come nel thread di background)

3. Le primitive fondamentali

Arc<T> — condividere la proprietà

`Arc` (Atomically Reference Counted) mette `T` sull'heap con un contatore di riferimenti thread-safe.

`Arc::clone(&x)` non copia il dato: copia solo il puntatore e incrementa il contatore. Il dato muore solo quando l'ultimo `Arc` viene droppato. Usalo ogni volta che più thread devono *possedere insieme* lo stesso dato.

Mutex<T> — mutua esclusione

`.lock().unwrap()` restituisce un `MutexGuard`: finché esiste, il lock è preso; quando esce di scope viene rilasciato automaticamente (nessun `unlock()` manuale). **Non è rientrante**: se lo stesso thread prova a prenderlo due volte, va in deadlock con se stesso.

ATTENZIONE

Tieni il lock per il minor tempo possibile: fai i calcoli "pesanti" fuori dal blocco protetto quando puoi, e non chiamare mai, mentre tieni un lock, un altro metodo che prova a riprendere lo stesso Mutex.

RwLock<T> — letture concorrenti, scritture esclusive

`.read()` permette a più thread di leggere insieme; `.write()` è esclusivo come un Mutex. Utile solo se le letture sono molto più frequenti delle scritture — altrimenti un `Mutex` normale è più semplice e altrettanto valido in un esame.

Tipi Atomici + Ordering

`AtomicBool`, `AtomicUsize`, `AtomicU64` ... per singoli valori semplici (flag di stop, contatori) senza il costo concettuale di un Mutex. Metodi chiave: `.load(Ordering::SeqCst)`, `.store(v, Ordering::SeqCst)`, `.fetch_add(1, Ordering::SeqCst)`. In un esame, `Ordering::SeqCst` ovunque è sempre una scelta sicura e giustificabile (è l'ordinamento più forte, "capisci tutto e basta").

mpsc::channel — comunicazione, non condivisione

Un `Sender<T>` (clonabile, multi-produttore) e un `Receiver<T>` (unico, non clonabile). `recv()` blocca finché arriva un messaggio o tutti i sender sono droppati; `recv_timeout(d)` blocca al massimo `d`. Trucco molto usato in questi esami: **riusare lo stesso canale sia come timer periodico sia come segnale di spegnimento** (vedi Pattern B2).

Condvar — aspettare una condizione, non un tempo

Va sempre abbinata a un `Mutex` che protegge la condizione stessa. `cvar.wait(guard)` rilascia il lock ed aspetta un `notify`; quando si risveglia riprende il lock. Va sempre chiamata **dentro un** `while`, mai un semplice `if`, per via dei risvegli spuri.

thread::spawn vs thread::scope

`thread::spawn(move || ...)` crea un thread che può vivere più a lungo della funzione chiamante: la closure deve possedere (`move`) tutto ciò che usa e tutto deve essere `'static` — da qui la necessità di `Arc`. `thread::scope(|s| { s.spawn(...); ... })` invece **garantisce** che tutti i thread finiscano prima che `scope` ritorni: puoi quindi passare riferimenti a variabili locali senza `Arc`. Usalo per il classico "fork-join" (lancia N thread, aspetta tutti, raccogli i risultati) quando non ti serve un thread che sopravviva alla funzione.

4. Sei pattern pronti all'uso

Codice-scheletro da adattare direttamente durante l'esame. Sono gli stessi mattoncini, ricombinati, dell'esercizio Aggregator.

Pattern A — Struct con stato condiviso e metodi &self

quando serve "thread-safe" su più chiamate

```
use std::sync::{Arc, Mutex};

pub struct Counter {
    inner: Arc<Mutex<u64>>, // Arc: condiviso; Mutex: mutabile in sicurezza
}

impl Counter {
    pub fn new() -> Self {
        Counter { inner: Arc::new(Mutex::new(0)) }
    }

    pub fn increment(&self) { // &self, NON &mut self
        let mut guard = self.inner.lock().unwrap();
        *guard += 1;
    } // guard droppato qui -> lock rilasciato

    pub fn get(&self) -> u64 {
        *self.inner.lock().unwrap()
    }
}
```

Pattern B1 — Thread periodico, stop con AtomicBool

semplice da spiegare; stop entro un periodo

```
use std::sync::atomic::{AtomicBool, Ordering};

let running = Arc::new(AtomicBool::new(true));
let running_bg = Arc::clone(&running);
let state_bg = Arc::clone(&shared_state);

let handle = std::thread::spawn(move || {
    while running_bg.load(Ordering::SeqCst) {
        std::thread::sleep(period);
        // ... lock, calcola snapshot, aggiorna stato ...
    }
});
// per fermarlo: running.store(false, Ordering::SeqCst); poi handle.join();
```

Pattern B2 — Thread periodico, stop con mpsc + recv_timeout

quando serve uno stop davvero immediato alla Drop

```
use std::sync::mpsc::{self, RecvTimeoutError};

let (stop_tx, stop_rx) = mpsc::channel::<()>();
let state_bg = Arc::clone(&shared_state);

let handle = std::thread::spawn(move || {
    loop {
        match stop_rx.recv_timeout(period) {
            Err(RecvTimeoutError::Timeout) => {
                // periodo scaduto: fai lo snapshot, come sopra
            }
            _ => break, // Sender droppato (Disconnected) o messaggio esplicito: fermati
        }
    }
});
// alla Drop: drop(stop_tx) sblocca SUBITO la recv_timeout (Disconnected), niente attesa
```

Pattern C — Drop che ferma e joina il thread

obbligatorio ogni volta che c'è un thread interno

```
pub struct Worker {
    // ...
    stop: Arc<AtomicBool>, // oppure: shutdown: Option<Sender<()>>
    handle: Option<std::thread::JoinHandle<()>>, // Option: serve per poterlo "portare via"
}

impl Drop for Worker {
    fn drop(&mut self) {
        self.stop.store(true, Ordering::SeqCst); // 1. SEGNALA lo stop
        if let Some(h) = self.handle.take() {    // 2. POI joina
            let _ = h.join();
        }
    }
}
```

Pattern D — Fork-join con `thread::scope`

più thread, un solo risultato aggregato, nessun thread da far sopravvivere

```
let totale: i32 = std::thread::scope(|s| {
    let handles: Vec<_> = dati.iter()
        .map(|chunk| s.spawn(move || elabora(chunk))) // può prendere &dati, niente
Arc
    .collect();
    handles.into_iter().map(|h| h.join().unwrap()).sum()
}); // qui thread::scope garantisce che TUTTI i thread siano già finiti
```

Pattern E — Produttore/consumatore con `mpsc`

"in coda", passaggio di messaggi invece di stato condiviso

```
let (tx, rx) = std::sync::mpsc::channel();

let tx2 = tx.clone(); // Sender è clonabile: più produttori
let producer = std::thread::spawn(move || {
    for i in 0..10 { tx2.send(i).unwrap(); }
}); // quando tx2 (e ogni altro clone) viene
droppato...

drop(tx); // ...e anche l'originale...
for ricevuto in rx { // ...rx smette di bloccare e il for termina da
    solo
        println!("{ricevuto}");
}
producer.join().unwrap();
```

Pattern F — Aspettare una condizione con `Condvar`

"aspetta finché non è pronto/disponibile", non solo "aspetta N ms"

```
use std::sync::{Mutex, Condvar};

pub struct Signal { ready: Mutex<bool>, cvar: Condvar }

impl Signal {
    pub fn wait_until_ready(&self) {
        let mut ready = self.ready.lock().unwrap();
        while !*ready { // SEMPRE while, mai if (risvegli
            spuri)
            ready = self.cvar.wait(ready).unwrap();
        }
    }
    pub fn set_ready(&self) {
        *self.ready.lock().unwrap() = true; // cambia lo stato PRIMA di notificare
        self.cvar.notify_all();
    }
}
```


5. Catalogo degli errori classici

1. Valore dinamico generato più volte quando serve una sola volta

✗ SBAGLIATO

```
for (id, b) in map.iter() {
  out.push(Average {
    id: *id,
    // Instant::now() diverso per OGNI
    // elemento del ciclo!
    t: Instant::now(),
    avg: b.avg(),
  });
}
```

✓ GIUSTO

```
let now = Instant::now(); // una volta sola
for (id, b) in map.iter() {
  out.push(Average {
    id: *id,
    t: now, // stesso valore
    avg: b.avg(),
  });
}
```

Capita ogni volta che i test confrontano fra loro più elementi generati "nello stesso giro logico" (stesso timestamp, stesso id di batch...): se lo generi dentro il loop invece che prima, ogni elemento avrà un valore leggermente diverso e i confronti di uguaglianza falliscono in modo intermittente. **Regola: se un valore deve essere "condiviso" fra più risultati dello stesso ciclo, calcolalo una volta sola, fuori dal ciclo.**

2. Segnalare lo stop dopo aver già fatto join

✗ SBAGLIATO — DEADLOCK

```
if let Some(h) = self.handle.take() {
  h.join().unwrap(); // aspetta per sempre:
} // il thread aspetta lo
self.stop.store(true, ..); // stop, che arriva DOPPO
```

✓ GIUSTO

```
self.stop.store(true, ..); // 1. segnala
if let Some(h) = self.handle.take() {
  h.join().unwrap(); // 2. poi aspetta
}
```

3. Dimenticare `move` nella closure di `thread::spawn`

Errore del compilatore tipico: `closure may outlive the current function... use the 'move' keyword` (E0373). Succede perché `thread::spawn` esige una closure `'static'`: senza `move` proverebbe a catturare per riferimento variabili locali che stanno per essere distrutte. Fix: `move ||` davanti alla closure, dopo aver preparato i clone di `Arc` necessari.

4. Passare direttamente l'`Arc` originale al posto di un clone

✗ SBAGLIATO

✓ GIUSTO

```
let handle = thread::spawn(move || {
    usa(shared_state); // shared_state
}); // ORIGINALE
consumato:
// dopo qui, shared_state non esiste
// più fuori dal thread!
```

```
let state_bg = Arc::clone(&shared_state);
let handle = thread::spawn(move || {
    usa(state_bg); // solo il CLONE è
}); // consumato dal
thread
// shared_state resta usabile qui fuori
```

5. Restituire un riferimento preso dentro un MutexGuard

Non puoi restituire da un metodo un `&T` ottenuto da dentro `self.stato.lock().unwrap()`: il `MutexGuard` viene distrutto a fine funzione (rilasciando il lock) e il riferimento non sopravvivrebbe — il borrow checker te lo impedisce. Soluzione standard: `.clone()` il dato prima di restituirlo (accettabile per dati piccoli come in questi esercizi).

6. Doppio lock dello stesso Mutex nello stesso thread

`Mutex` in Rust non è rientrante: se un metodo che tiene il lock chiama (direttamente o indirettamente) un altro metodo che rifà `.lock()` sullo **stesso** Mutex, il thread si blocca in attesa di se stesso (deadlock silenzioso, nessun panic, il programma si impalla e basta). Attenzione in particolare a metodi che chiamano altri metodi pubblici della stessa struct.

7. Ordine di lock incoerente fra due Mutex

Se una struct ha due Mutex distinti (es. uno per i dati, uno per le statistiche) e in punti diversi del codice li prendi in ordine diverso (A poi B in un metodo, B poi A in un altro), due thread possono bloccarsi a vicenda. Regola pratica da esame: **stabilisci un ordine fisso e rispettalo sempre**, oppure — più semplice — accorpa i dati correlati in un solo Mutex (come fa `SharedState` nell'esempio finale).

6. Checklist operativa passo-passo

- Sottolinea nel testo le parole-spia (tabella §1) e scrivi a fianco quale primitiva implicano.
- Guarda la firma dei metodi nello stub: `&self` conferma interior mutability.
- Leggi **tutti** i test forniti prima di scrivere l'implementazione: ricava da lì la semantica esatta (quando un dato appare/sparisce, quali valori devono coincidere, quanto durano gli `sleep`).
- Disegna a penna lo stato condiviso: che campi, che tipo, chi legge, chi scrive, quanto spesso.
- Percorri il decision tree (§2) e scegli le primitive.
- Se c'è un thread interno: scrivi **subito** anche `impl Drop` (Pattern C) — è la parte più facile da dimenticare, e senza non c'è "terminazione sicura".
- Implementa i metodi pubblici tenendo il lock il minor tempo possibile.
- `cargo build` spesso: gli errori sui thread sono quasi sempre "manca un `Arc::clone`" o "manca `move`".
- `cargo test` e leggi bene i messaggi di panic: spesso indicano subito il campo che non torna.
- Rileggi il testo un'ultima volta per i casi limite espliciti (stato vuoto, "solo se ha inviato almeno un dato", ecc.) e verifica di avere un test — anche mentale — per ciascuno.

7. Esempio completo commentato — Aggregator

Traccia: un Aggregator riceve misure di temperatura da più sensori/thread, e ogni `sample_time_millis` deve calcolare la media per sensore relativa alle misure ricevute in quel periodo. Ecco come si mappa sui pattern precedenti.

REQUISITO NEL TESTO	PATTERN APPLICATO
"metodi thread-safe" <code>add_measure</code> / <code>get_averages</code>	Pattern A — <code>Arc<Mutex<SharedState>></code> , metodi <code>&self</code>
"campionamento ad intervalli regolari"	Pattern B1 — thread con <code>while running.load() {{ sleep(...); ... }}</code>
"alla distruzione, il thread deve terminare in modo sicuro"	Pattern C — <code>Option<JoinHandle></code> + <code>Drop</code>
"più thread possono inviare misure contemporaneamente"	Stesso <code>Mutex</code> per tutti: le scritture si serializzano da sole

SRC/MAIN.RS — STATO CONDIVISO

```
pub struct Bucket { pub sum: f64, pub count: usize }

pub struct SharedState {
    pub map: HashMap<usize, Bucket>,    // accumulo del periodo IN CORSO
    pub last_averages: Vec<Average>,    // fotografia dell'ULTIMO periodo chiuso
}
```

I due campi stanno nello **stesso** Mutex apposta: "chiudere un periodo" significa leggere `map`, scrivere `last_averages` e svuotare `map` in un solo blocco atomico — se fossero due Mutex separati, un `get_averages()` potrebbe intercettare uno stato a metà.

COSTRUTTORE — AVVIO DEL THREAD DI BACKGROUND

```

pub fn new(sample_time_millis: u64) -> Self {
    let shared_state = Arc::new(Mutex::new(SharedState { map: HashMap::new(), last_averages:
Vec::new() }));
    let running = Arc::new(AtomicBool::new(true));

    let worker_handle = {
        let shared_state = Arc::clone(&shared_state);    // clone DEDICATO al thread
        let running = Arc::clone(&running);              // (shadowing: fuori restano gli
originali)
        std::thread::spawn(move || {                    // move: la closure ne diventa
proprietaria
            while running.load(Ordering::SeqCst) {
                std::thread::sleep(Duration::from_millis(sample_time_millis));
                let mut state = shared_state.lock().unwrap();
                let now = Instant::now();                // <- una volta sola, FUORI dal
for
                let mut averages = Vec::new();
                for (sensor_id, bucket) in state.map.iter() {
                    if bucket.count > 0 {
                        averages.push(Average {
                            sensor_id: *sensor_id,
                            reference_time: now,
                            average_temperature: bucket.sum / bucket.count as f64,
                        });
                    }
                }
                state.last_averages = averages;
                state.map.clear();                      // periodo chiuso: si riparte da
zero
            }
        })
    };

    Aggregator { shared_state, running, worker_handle: Some(worker_handle) }
    // ^ new() ritorna SUBITO qui: non aspetta affatto il thread, che continua a girare da
solo
}

```

IL PUNTO CHE CONFONDE DI PIÙ

`thread::spawn` è "fire and forget": crea il thread e ritorna subito un `JoinHandle`, senza aspettare nulla. `new()` finisce in pratica nella stessa frazione di microsecondo in cui il thread parte — non "dopo" che il thread ha girato o si è fermato. Il thread vive per conto suo finché qualcuno non chiama `.join()` su quell'handle, cosa che qui avviene solo dentro `Drop`.

ADD_MEASURE / GET_AVERAGES

```

pub fn add_measure(&self, sensor_id: usize, temperature: f64) {
    let mut state = self.shared_state.lock().unwrap();
    let bucket = state.map.entry(sensor_id).or_insert(Bucket { sum: 0.0, count: 0 });
    bucket.sum += temperature;
    bucket.count += 1;
}

pub fn get_averages(&self) -> Vec<Average> {
    self.shared_state.lock().unwrap().last_averages.clone() // clone: il guard non può
    "uscire"
}

```

DROP — SPEGNIMENTO SICURO

```

impl Drop for Aggregator {
    fn drop(&mut self) {
        self.running.store(false, Ordering::SeqCst); // 1. segnala lo stop
        if let Some(handle) = self.worker_handle.take() {
            let _ = handle.join(); // 2. aspetta che finisca davvero
        }
    }
}

```

Nota sui tempi: se `running` diventa `false` mentre il thread sta dormendo, quello non se ne accorge subito (`sleep` non è interrompibile); si sveglia, fa un ultimo giro "a vuoto" e poi esce dal `while`. Lo spegnimento richiede quindi al massimo `sample_time_millis` — accettabile: "sicuro" nel testo dell'esame vuol dire "senza panic/leak/deadlock", non "istantaneo".

8. Scheda di riferimento rapido

Arc<T>

Condivide la proprietà di T fra thread. `Arc::clone` copia il puntatore, non il dato.

Mutex<T>

`.lock().unwrap()` → `MutexGuard`; si rilascia da solo a fine scope. Non rientrante.

RwLock<T>

`.read()` multiplo, `.write()` esclusivo. Solo se le letture dominano.

AtomicBool/AtomicUsize/...

`.load(Ordering::SeqCst)` / `.store(...)`. Per flag e contatori semplici, senza Mutex.

mpsc::channel

`Sender` clonabile, `Receiver` unico. `recv_timeout` = timer + segnale di stop in uno.

Condvar

Sempre con un Mutex; `wait` dentro un `while`, mai un `if`.

thread::spawn

Ritorna subito un `JoinHandle`, non aspetta nulla. Richiede closure `move` e `'static`.

thread::scope

Aspetta tutti i thread figli prima di ritornare: permette riferimenti a dati locali, niente Arc.

Drop + thread interno

Sempre: `Option<JoinHandle>`, segnala lo stop, poi `.take().join()` — mai al contrario.

Valori dinamici condivisi (timestamp, id...)

Calcolali **una volta sola** prima di un ciclo che li riusa su più elementi correlati.